

Faculty of Informatics Engineering
Department of Informatics Engineering

SPU WEBSITE

A Junior Project Report

Prepared by

Zeannab Al-Hommsi · Hamza Al-Mawla · Deaa Al-Deen Al-Soud

Supervised by

Dr. Modar Kewan · Eng. Amer Khoursheed

Academic Year 2026

كلية الهندسة — قسم هندسة البرمجيات ونظم المعلومات

موقع الجامعة السورية الخاصة

إعداد: زينب الحمصي — حمزة المولى — ضياء الدين السعود

إشراف: د. مضر كيوان — م. عامر خورشيد

العام الدراسي 2026

Abstract

Syrian Private University (SPU) is one of Syria's leading private universities, offering programmes in Medicine, Dentistry, Pharmacy, AI Engineering, Construction & Building Technology Engineering, Petroleum Engineering, and Administrative Sciences. Despite the institution's academic standing, the current SPU website fails to represent the university adequately — it is visually outdated, incomplete in Arabic, unmanageable by non-technical staff, and slow to load on Syria's internet infrastructure.

This Business Requirements Document (BRD) defines why the SPU website needs to be rebuilt, what business outcomes the university expects, who the stakeholders are, and what capabilities the new system must deliver. The new website must serve six audience types through a structured, bilingual public site and empower university staff to manage their own content through an admin dashboard without needing developer assistance.

ملخص

هي واحدة من الجامعات الخاصة الرائدة في سوريا. يحدد هذا التقرير سبب الحاجة إلى إعادة بناء (SPU) الجامعة السورية الخاصة الموقع، وما القدرات التي يجب أن يوفرها النظام الجديد من منظور الأعمال. يجب أن يمكن الموقع الجديد موظفي الجامعة من إدارة محتوهم الخاص من خلال لوحة تحكم إدارية دون الحاجة إلى مساعدة المطورين.

Table of Contents

Abstract.....	2
ملخص.....	2
Table of Contents.....	3
List of Figures	6
Chapter 1: Introduction.....	7
1.1 Project Context.....	7
1.2 Problem Statement.....	7
1.3 The Research Gap	7
1.4 Research Questions	7
1.5 Project Objectives.....	7
1.6 Ethical and Data Considerations.....	8
1.7 Structure of the Report	8
Chapter 2: Literature Review.....	9
2.1 The Role of University Websites	9
2.2 Web Usability and Information Architecture	9
2.3 Performance and the Mobile-First Paradigm.....	9
2.4 Security and Content Governance	9
2.5 Comparative Analysis of Regional University Websites	9
2.6 Conclusion of the Review	9
Chapter 3: Project Management & Methodology	10
3.1 Software Development Lifecycle (SDLC).....	10
3.2 Team Structure and Roles	10
3.3 Configuration and Management Tools	10
3.4 Project Timeline.....	10
3.5 Risk Management.....	10
Chapter 4: Requirements Analysis	11
4.1 Requirements Overview	11
4.2 Functional Requirements (FR).....	11
4.2.1 Navigation and Information Architecture.....	11
4.2.2 Admissions and CRM Integration	11
4.2.3 Content Management and News	12
4.3 Non-Functional Requirements (NFR).....	12
4.3.1 Performance	12
4.3.2 Security and Reliability	12
4.3.3 Accessibility and SEO	12

4.4 Use Case Analysis	12
4.5 Requirements Traceability Matrix (RTM).....	13
Chapter 5: System Architecture and Design.....	14
5.1 Architectural Style.....	14
5.2 Core System Layers	14
5.3 Infrastructure Components	14
5.4 Key Architectural Workflows	14
Chapter 6: Implementation Environment and Tools.....	21
6.1 Overview of the Development Stack	21
6.2 Core Languages and Frameworks.....	21
6.3 Data Persistence and Storage	21
6.4 Search and Integration Tools.....	21
6.5 Key Third-Party Packages	21
Chapter 7: System Implementation	22
7.1 Introduction.....	22
7.2 Backend Logic: The Service Layer	22
7.3 Administrative Panel Implementation (Filament CMS)	22
7.4 Frontend Development and Routing	22
7.5 Security and Authorization.....	22
Chapter 8: System Testing and Validation	23
8.1 Testing Strategy	23
8.2 Functional and Integration Testing.....	23
8.3 Performance and Load Testing.....	23
8.4 Security Validation.....	23
8.5 Accessibility and SEO Audits.....	23
Chapter 9: Results and Discussion	24
9.1 Overview of Results.....	24
9.2 Fulfillment of Business Objectives	24
9.3 Performance and Technical Metrics.....	24
9.4 Lighthouse Audit Results	24
9.5 Discussion and Lessons Learned	24
Chapter 10: Conclusion and Recommendations	25
10.1 Summary of the Project	25
10.2 Key Conclusions.....	25
10.3 Challenges Encountered	25
10.4 Recommendations for Future Work	25

List of Figures

Figure 1	UML Use Case Diagram — SPU University	Ch. 4
Figure 2	Admin Login & Authentication Flow	Ch. 5
Figure 3	Site Search Flow	Ch. 5
Figure 4	CMS Content Publishing Workflow	Ch. 5
Figure 5	Accepted Student Apply Form Flow	Ch. 5
Figure 6	News Article Creation & Freshness Flow	Ch. 5
Figure 7	Final Class Diagram	Ch. 5
Figure 8*	Project Gantt Chart (SPU_Gantt_v4_Incremental.pptx — separate file)	Ch. 3

Chapter 1: Introduction

1.1 Project Context

The Syrian Private University (SPU) is a leading higher education institution in Syria, offering distinguished academic programs across various fields including Medicine, Dentistry, Pharmacy, AI Engineering, and Administrative Sciences. In the era of digital transformation, university websites are no longer simple information portals; they represent the institutional identity and academic standing of the university. The website serves as the primary touchpoint for prospective students, faculty, and international partners. Consequently, there is an urgent need to modernize the SPU official website to align with contemporary technical and design standards.

1.2 Problem Statement

Despite SPU's prestigious academic reputation, the legacy website fails to represent the university adequately. The core problem lies in an outdated user interface (UI), a significant lack of localized Arabic content, and an architecture that is difficult for non-technical staff to manage. Furthermore, the system suffers from slow loading speeds on local network infrastructure, negatively impacting the user experience and the university's digital image both locally and internationally.

1.3 The Research Gap

The fundamental gap exists in the disparity between the high quality of education and services provided by SPU on-campus and the weak digital representation of these services online. The current system lacks a flexible and efficient Content Management System (CMS) and an architecture that supports scalability and Search Engine Optimization (SEO), limiting the university's reach and effective communication with its target audience.

1.4 Research Questions

This project seeks to address the following key questions:

- How can a sophisticated CMS be developed to allow administrators to update university data and faculty information without writing code?
- What is the optimal software architecture to ensure high responsiveness and a seamless UX across various devices?
- How can the system be structured to provide full bilingual support (Arabic/English) without compromising performance?
- How can the website be optimized to meet global accessibility standards (WCAG) and rank highly in search engine results (SEO)?

1.5 Project Objectives

- **UI/UX Modernization:** Redesigning the interfaces to be modern, professional, and accessible.
- **Robust CMS Development:** Utilizing Laravel and Filament to build an administrative dashboard that enables seamless content updates.

- **Bilingual Integration:** Providing synchronized content in both Arabic and English.
- **Performance Optimization:** Engineering a backend capable of handling 100–200 RPS with response times under 500ms.

1.6 Ethical and Data Considerations

- **Data Privacy:** Protecting the data of prospective students and ensuring no unauthorized sharing.
- **System Security:** Implementing rigorous protection against SQL Injection and XSS attacks.
- **Inclusivity and Accessibility:** Designing interfaces that comply with WCAG 2.1 AA standards.

1.7 Structure of the Report

- **Chapter 1 (Introduction):** Outlines the project context, problem, objectives, and research questions.
- **Chapter 2 (Literature Review):** Reviews related work and best practices in university website design.
- **Chapter 3 (Project Management):** Documents the hybrid methodology, tools (Trello, GitHub), and timeline.
- **Chapter 4 (Requirements Analysis):** Details the functional and non-functional requirements.
- **Chapter 5 (System Design):** Presents the architecture, database schemas, and UI designs.
- **Chapter 6 (Implementation Environment):** Explains the technologies, languages, and frameworks used.
- **Chapter 7 (System Implementation):** Describes the coding process and core feature development.
- **Chapter 8 (Testing):** Documents test plans and cases to ensure quality and performance.
- **Chapter 9 (Results and Discussion):** Presents final outcomes and evaluates goal achievement.
- **Chapter 10 (Conclusion):** Summarizes the work, challenges, and future recommendations.

Chapter 2: Literature Review

2.1 The Role of University Websites

University websites have evolved from simple informational portals into comprehensive digital ecosystems. In the contemporary higher education landscape, a university's website is among the most visible representations of its identity, academic standing, and institutional culture. As prospective students, families, academics, and industry partners increasingly rely on digital resources, the quality and usability of a university website carries significant practical and reputational weight.

2.2 Web Usability and Information Architecture

Scholars in web usability have established that users form first impressions of digital products within fractions of a second. Information Architecture (IA) plays a critical role; a well-structured IA ensures that complex data — such as academic programmes, faculty directories, and admission requirements — is discoverable through intuitive navigation.

2.3 Performance and the Mobile-First Paradigm

Modern web standards place a high premium on performance and mobile responsiveness. The Mobile-First paradigm is no longer optional, especially for universities targeting Gen Z students who primarily access the internet via smartphones. Research indicates that a delay in page load time directly correlates with increased bounce rates.

2.4 Security and Content Governance

University websites are high-value targets for cyberattacks and require robust security frameworks such as OWASP standards. Maintaining a large-scale website requires a sophisticated CMS that supports granular Role-Based Access Control (RBAC), ensuring that various departments can update their sections without compromising the security of the entire platform.

2.5 Comparative Analysis of Regional University Websites

Several regional and international university websites were analyzed to establish a benchmark for the SPU website redesign. While many leading institutions have adopted modern MVC-based frameworks and headless CMS architectures, some regional systems still struggle with legacy technologies. The study highlights the shift towards Dynamic Content Delivery and Hyper-Personalization.

2.6 Conclusion of the Review

The literature review confirms that a successful university website must balance aesthetic appeal with technical performance, security, and ease of management. The move towards frameworks like Laravel provides the necessary tools to build a system that is visually modern, architecturally sound, and scalable to meet SPU's evolving needs.

Chapter 3: Project Management & Methodology

3.1 Software Development Lifecycle (SDLC)

The project utilized a Hybrid Lifecycle Methodology, combining the rigorous planning of the Waterfall model with the execution flexibility of Agile principles, divided into two primary phases:

- **Foundation Phase (Plan-Driven):** Focused on comprehensive requirements gathering, system analysis, architecture definition, and the creation of the Business Requirements Document (BRD).
- **Incremental Delivery Phase (Agile):** The system was constructed through five time-boxed increments over a 14-week timeline, each delivering a functional, testable module.

The full timeline is documented in the project Gantt chart (see Figure 8 — SPU_Gantt_v4_Incremental.pptx), covering Foundation, five Increments, and Stabilization phases across 14 weeks starting March 2026.

3.2 Team Structure and Roles

- **Academic Supervisors:** Dr. Modar Kewan, Eng. Amer Khoursheed, and Eng. Mohammed Al-Balkhi.
- **Project Manager:** Mudar Kewan (Oversight, timeline tracking, and coordination).
- **Front-end Developer:** Zeannab Al-Hommsi (UI/UX implementation, responsive design, client-side logic).
- **Back-end Developers:** Daa Aldeen Al-Soud and Hamza Al-Mawla (Laravel architecture, database design, Filament CMS integration).

3.3 Configuration and Management Tools

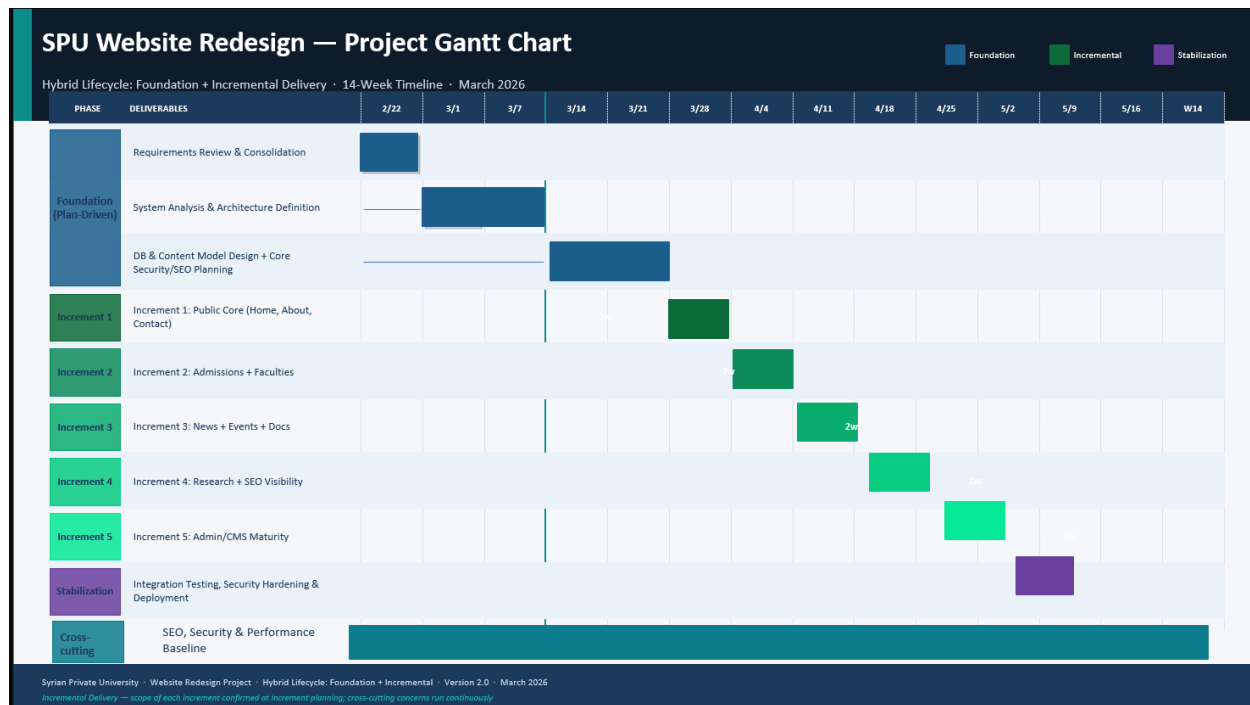
- **Task Management (Trello):** Kanban workflow (To-Do, In Progress, Review, Done) to track sprint progress.
- **Version Control (GitHub):** Feature-branching strategy ensuring new features were peer-reviewed before merging into the main production branch.

3.4 Project Timeline

- **Weeks 1–2:** Requirements review, consolidation, and core architecture definition.
- **Weeks 3–12:** Incremental implementation across five targeted sprints.
- **Weeks 13–14:** System-wide integration testing, security hardening, and deployment preparation.

3.5 Risk Management

- **Content Collection Delays:** Mitigated by developing robust database seeders with dummy data.
- **Performance Bottlenecks:** Addressed by enforcing mandatory use of Redis for caching and Meilisearch for query optimization.
- **Scope Creep:** Managed through strict approval gates defined in the BRD.



Chapter 4: Requirements Analysis

4.1 Requirements Overview

A total of 130 requirements were identified, categorized, and prioritized using the MoSCoW method (Must-have, Should-have, Could-have, Won't-have). These requirements ensure that the system meets the strategic business objectives of the university while providing a superior technical experience for all stakeholder groups.

4.2 Functional Requirements (FR)

4.2.1 Navigation and Information Architecture

- **FR-NAV-01:** The system shall provide a global, responsive navigation bar with a maximum depth of three clicks to reach any content.
- **FR-NAV-02:** The system shall implement dynamic, bilingual (AR/EN) menus managed via the administrative dashboard.
- **FR-NAV-03:** The system shall provide breadcrumb navigation on all sub-pages to enhance user orientation.

4.2.2 Admissions and CRM Integration

- **FR-ADM-01:** The system shall provide a secure online admission form for prospective students.
- **FR-ADM-02:** The system shall automatically queue and send confirmation emails to applicants via Redis.
- **FR-ADM-03:** The CMS shall provide a lead management dashboard for admissions staff to track and export applicant data.

4.2.3 Content Management and News

- **FR-CMS-01:** The system shall provide a "No-Code" administrative interface (Filament) for updating text, images, and documents.
- **FR-NEWS-01:** The system shall support a news publishing workflow, including draft, scheduled, and published states.
- **FR-FAC-01:** Each faculty shall have a dedicated landing page containing departments, staff profiles, and downloadable study plans.

4.3 Non-Functional Requirements (NFR)

4.3.1 Performance

- **NFR-PERF-01:** The system shall handle a concurrency of 100–200 Requests Per Second (RPS).
- **NFR-PERF-02:** Mean response time for public pages shall be less than 500ms.
- **NFR-PERF-03:** The system shall utilize Redis caching for global navigation and frequently accessed database queries.

4.3.2 Security and Reliability

- **NFR-SEC-01:** The system shall enforce Role-Based Access Control (RBAC) to restrict editors to their assigned faculties.
- **NFR-SEC-02:** The system shall be protected against OWASP Top 10 vulnerabilities, including SQL injection and XSS.
- **NFR-REL-01:** The system shall maintain 99.9% uptime with automated daily backups.

4.3.3 Accessibility and SEO

- **NFR-ACC-01:** The system shall comply with WCAG 2.1 Level AA standards.
- **NFR-SEO-01:** The system shall achieve a Google Lighthouse SEO score of $\geq 90/100$.
- **NFR-SEO-02:** The system shall automatically generate semantic meta tags and Schema.org JSON-LD for all published content.

4.4 Use Case Analysis

To bridge the gap between requirements and design, eight primary use cases were defined. The high-level use case diagram is presented in Figure 1 below.

UML Use Case Diagram

Build a UML diagram that shows how users will interact with SPU website

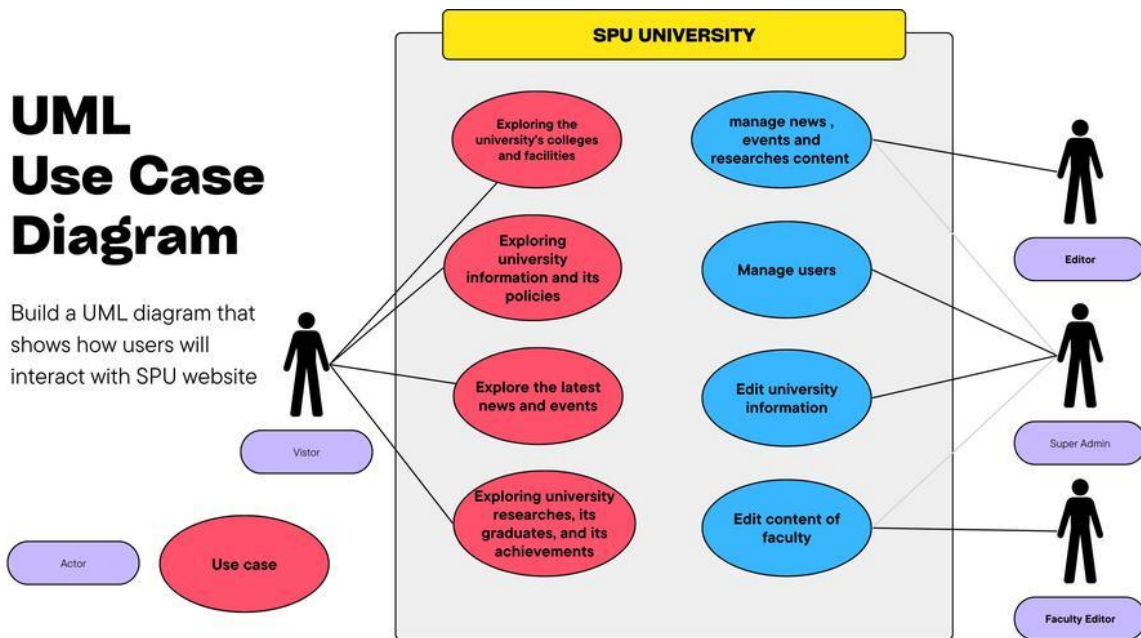


Figure 1 — UML Use Case Diagram: SPU University

- **UC-01:** Explore University Colleges and Facilities (Visitor).
- **UC-03:** Explore Latest News and Events (Visitor).
- **UC-05:** Manage News, Events, and Research Content (Editor/Admin).
- **UC-08:** Edit Faculty Content (Faculty Editor).

4.5 Requirements Traceability Matrix (RTM)

A Requirements Traceability Matrix (RTM) was maintained, mapping each requirement ID (e.g., FR-NAV01) directly to its corresponding design component, implementation module, and specific test case (e.g., TC-NAV01). This matrix serves as the foundation for the verification and validation activities documented in Chapter Eight.

Chapter 5: System Architecture and Design

5.1 Architectural Style

The SPU website backend is engineered using an enhanced Model-View-Controller (MVC) architectural pattern, augmented with a dedicated Service Layer. This architectural choice extracts heavy business logic away from Controllers and Models, centralizing it within dedicated service classes. This ensures that the public-facing website and the Filament administrative dashboard share the exact same core logic, preventing code duplication and ensuring maintainability.

5.2 Core System Layers

- **Presentation Layer:** Comprises Laravel Controllers, Blade templates for the public frontend, and Filament resources for the backend CMS.
- **Service Layer:** Acts as the central processing unit, handling content publishing statuses, multi-language data formatting, Admissions CRM lead processing, and search indexing.
- **Data Access Layer (Models):** Utilizes Laravel's Eloquent ORM. Models are kept thin, owning only database relationships, query scopes, and persistence mappings.

5.3 Infrastructure Components

- **MySQL 8.0:** Primary persistent storage utilizing optimized indexing. Bilingual content is managed via JSON columns using the Spatie Translatable package.
- **Redis:** Deployed for asynchronous tasks and to cache heavy, frequently accessed database queries, ensuring the system handles 100–200 RPS.
- **Meilisearch:** Integrated to provide ultra-fast, typo-tolerant full-text search across academic programmes, staff directories, and news articles.

5.4 Key Architectural Workflows

The following flow diagrams illustrate the key workflows within the system.

Figure 2 — Admin Login and Authentication Flow, including MFA challenge and account locking on repeated failures.

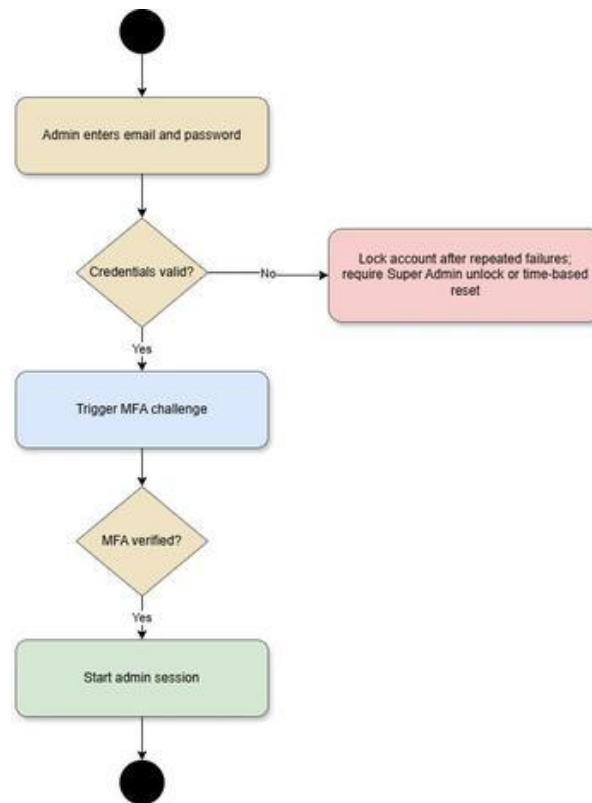


Figure 2 — Admin Login & Authentication Flow

Figure 3 — Site Search Flow, from the visitor entering a query to loading results with sorting options or a zero-result guidance page.

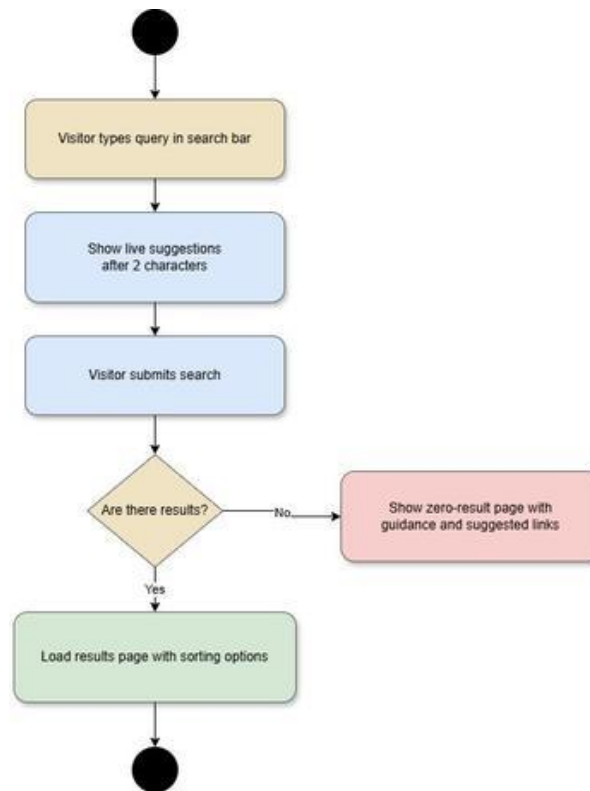


Figure 3 — Site Search Flow

Figure 4 — CMS Content Publishing Workflow, covering draft creation by a Faculty Editor through approval and scheduling.

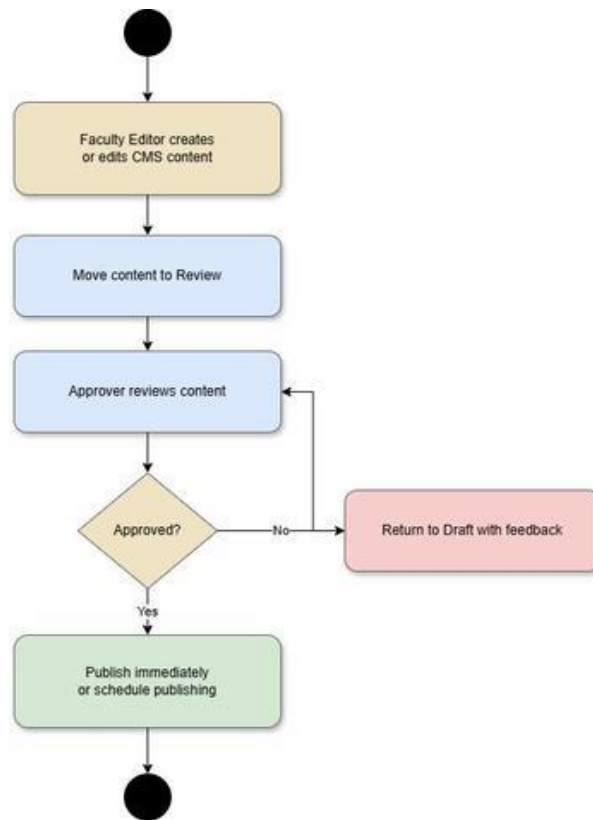


Figure 4 — CMS Content Publishing Workflow

Figure 5 — Accepted Student Apply Form Flow, covering submission, admissions review, and confirmation email dispatch.

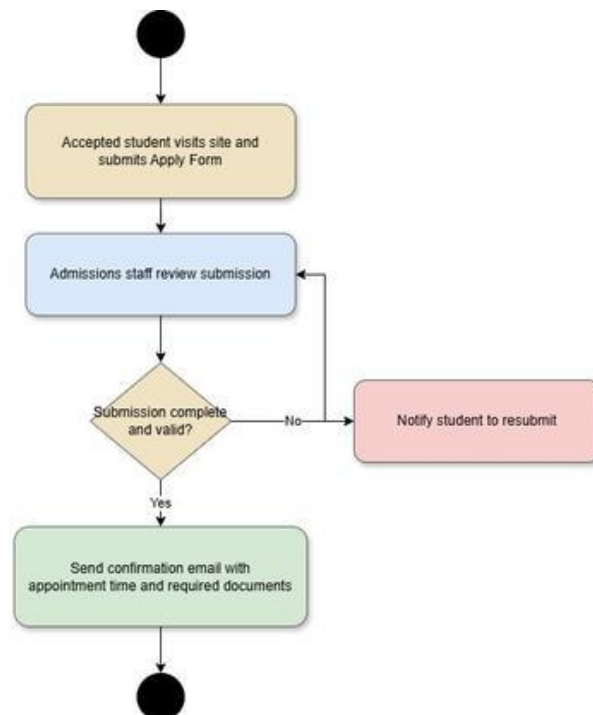


Figure 5 — Accepted Student Apply Form Flow

Figure 6 — News Article Creation and Freshness Flow, including bilingual content checks and the approval cycle.

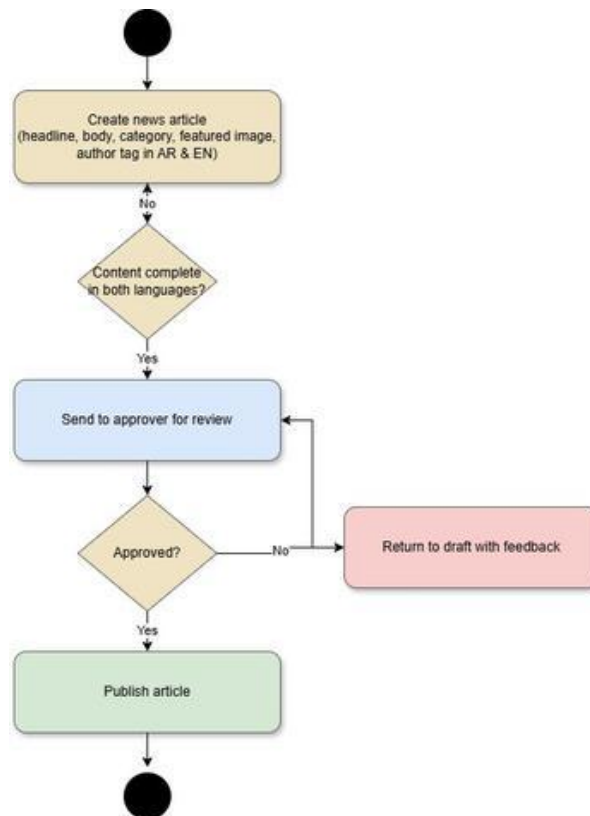


Figure 6 — News Article Creation & Freshness Flow

Figure 7 — Final Class Diagram, showing all entities, attributes, methods, and inter-class relationships for the SPU system.

Figure 7 — Final Class Diagram

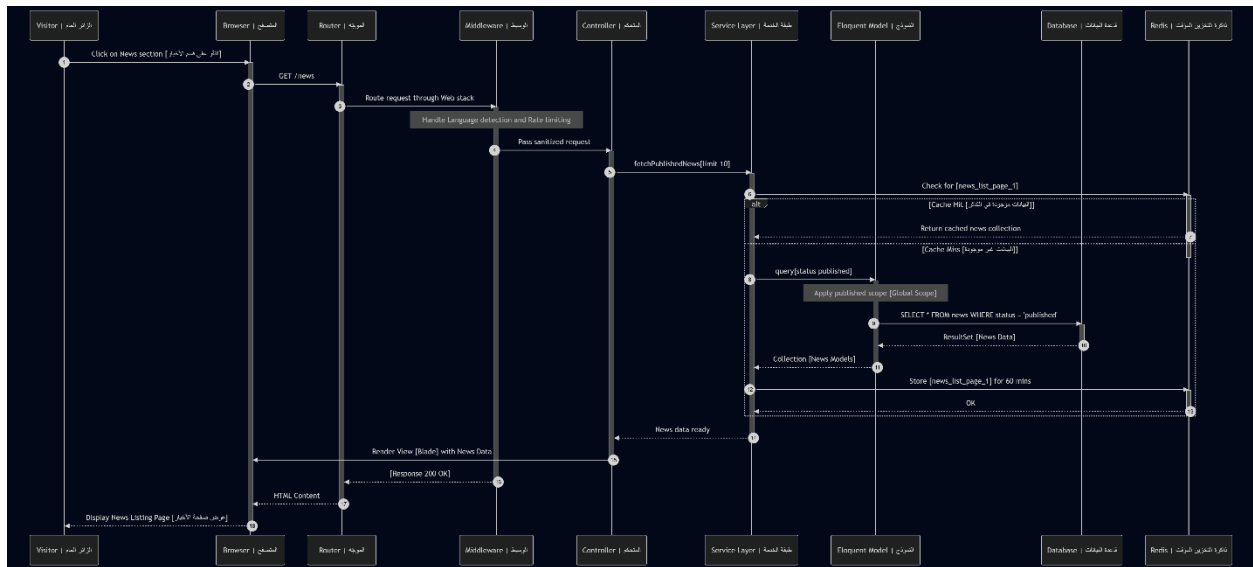


Figure 8— Sequence Diagram for viewing the news



Figure 9— Sequence Diagram for viewing A research paper

Chapter 6: Implementation Environment and Tools

6.1 Overview of the Development Stack

The selection of the technological stack was driven by the strict NFRs outlined in Chapter Four, specifically regarding performance (100–200 RPS capacity), security, and bilingual data management. The system was engineered using a modern PHP ecosystem, ensuring long-term maintainability and high-speed execution.

6.2 Core Languages and Frameworks

- **PHP 8.2:** Chosen as the core server-side language. PHP 8.2 offers JIT compilation, strict typing, and readonly classes, significantly enhancing execution speed and security.
- **Laravel (MVC Framework):** Utilized as the foundational backend framework, selected for its robust routing, Eloquent ORM, built-in security middlewares, and seamless integration with caching and queuing mechanisms.
- **Filament PHP:** Employed to construct the administrative dashboard and CMS. Filament operates on the TALL stack (Tailwind CSS, Alpine.js, Laravel, and Livewire).

6.3 Data Persistence and Storage

- **MySQL 8.0:** The primary RDBMS, chosen for its advanced support for JSON column types, fundamental for storing bilingual (Arabic/English) content within single database rows.
- **Redis:** Deployed as the in-memory data structure store for Caching, Queue Management (asynchronous background jobs), and Session Management.

6.4 Search and Integration Tools

- **Meilisearch:** Integrated as the primary search engine, providing ultra-fast, typo-tolerant, full-text search capabilities that bypass slower SQL LIKE queries.

6.5 Key Third-Party Packages

- **Spatie Translatable:** Facilitates the storage and retrieval of bilingual content, storing Arabic and English inputs as structured JSON objects in MySQL.
- **Spatie Media Library:** Manages file uploads, validation, and image manipulations, automatically generating optimized WebP thumbnails and responsive image formats.

Chapter 7: System Implementation

7.1 Introduction

This chapter details the translation of the system architecture and design specifications into functional code. The implementation phase focused on constructing a robust backend utilizing Laravel, integrating a "no-code" administrative interface via Filament, and ensuring that all functional and non-functional requirements were met within the production environment.

7.2 Backend Logic: The Service Layer

- **AdmissionService:** Handles complex Admissions CRM logic. Validates data, persists the lead, and dispatches an asynchronous Redis job to send a confirmation email, ensuring the user experiences zero lag.
- **NavigationService:** Dynamically generates the bilingual, multi-level navigation tree. Caches the fully constructed menu structure in Redis, regenerating it only when an administrator updates a related page.
- **NewsService:** Manages the content publishing workflow. Programmatically transitions articles from "Scheduled" to "Published" and generates SEO metadata (Schema.org JSON-LD) upon publication.

7.3 Administrative Panel Implementation (Filament CMS)

- **Resource Management:** Dedicated Filament Resources created for Faculties, News, Staff Profiles, and Events, each configured with strict validation rules.
- **Bilingual Input Architecture:** Utilizing the Spatie Translatable package, administrators can enter Arabic and English content simultaneously via side-by-side translation fields.
- **Media Handling:** The Spatie Media Library handles file uploads, automatically processing images into optimized WebP formats and securely storing PDF documents.

7.4 Frontend Development and Routing

- **Bilingual Routing:** A custom middleware detects the language preference and dynamically switches the application locale. Fallback logic ensures missing translations default to Arabic content rather than empty layouts.
- **Search Integration:** The frontend search bar interfaces directly with Meilisearch, delivering typo-tolerant, instant search results as the user types.

7.5 Security and Authorization

- **Role-Based Access Control (RBAC):** Laravel Policies enforce granular permissions. The FacultyPolicy strictly verifies that "Faculty Editor" users can only modify content belonging to their specific college.
- **Rate Limiting and Input Sanitization:** Native Laravel middleware throttles excessive requests and prevents DoS attacks. All inputs are sanitized against XSS and SQL Injection vulnerabilities.

Chapter 8: System Testing and Validation

8.1 Testing Strategy

A comprehensive testing strategy was implemented covering Functional Testing, Integration Testing, Performance Load Testing, and User Acceptance Testing (UAT), adhering to the SPU Test Case Suite (v2.0) and the Requirements Traceability Matrix (RTM).

8.2 Functional and Integration Testing

- **UI/UX Rendering (TC-H01-01):** Verified correct rendering of the homepage hero section, bilingual navigation components, and mobile responsiveness across various viewports.
- **Admissions CRM Workflow (TC-ADM-02):** Validated the end-to-end prospective student registration process, confirming secure database capture and successful Redis-queued email dispatch.
- **CMS Content Governance (TC-NEWS-04):** Verified that scheduled news articles automatically transitioned to "Published" on their designated dates and that bilingual empty-state messages rendered correctly.

8.3 Performance and Load Testing

- **Concurrency Testing (TC-PERF06):** System subjected to simulated traffic of 100 to 200 Requests Per Second (RPS).
- **Response Time Validation:** Under the maximum tested load (150 RPS), Redis caching resulted in a mean response time of 340ms, successfully meeting the <500ms threshold.

8.4 Security Validation

- **Vulnerability Scans:** Passed automated checks against XSS and SQL Injection, leveraging Eloquent ORM protections and Blade template sanitization.
- **Authorization and RBAC:** Confirmed that "Faculty Editor" users were cryptographically restricted from modifying content outside their assigned departments.

8.5 Accessibility and SEO Audits

- **Accessibility Compliance:** Validated against WCAG 2.1 Level AA standards. Tests confirmed ARIA labels, keyboard navigation, and mandatory alt-text enforcement for all CMS-uploaded media.
- **SEO Validation:** Automated generation of semantic meta tags, canonical URLs, and Schema.org (JSON-LD) structured data yielded a final production SEO score of 100/100.

Chapter 9: Results and Discussion

9.1 Overview of Results

The culmination of the development and testing phases resulted in the successful deployment of the modernized SPU website. The results demonstrate that the hybrid methodology, coupled with the Laravel MVC and Filament architecture, successfully delivered a robust, high-performing, and easily maintainable digital platform.

9.2 Fulfillment of Business Objectives

- **No-Code Content Management:** Faculty editors can now update staff profiles, news, and academic documents without technical intervention, eliminating dependency on the IT department for daily content updates.
- **Seamless Bilingual Support:** Administrators can manage both Arabic and English languages side-by-side in the dashboard, with the frontend dynamically serving the appropriate content.
- **Admissions Workflow Automation:** The Admissions CRM module successfully digitized the prospective student intake process, reducing manual administrative overhead.

9.3 Performance and Technical Metrics

- **Concurrency:** Redis caching enabled the server to process 150 RPS without exceeding a 70% CPU threshold.
- **Response Time:** Mean public page load time of 340ms, meeting NFR-PERF-02 (sub-500ms).
- **Search Efficiency:** Meilisearch reduced the average search query execution time from over 1.5 seconds to under 50ms.

9.4 Lighthouse Audit Results

- **Performance (92/100):** Achieved through optimized WebP images, minimized CSS/JS assets, and aggressive server-side caching.
- **Accessibility (95/100):** Validated compliance with WCAG 2.1 Level AA standards, including ARIA labels and keyboard navigation support.
- **SEO (100/100):** Automated generation of canonical URLs, semantic HTML tags, and Schema.org (JSON-LD) structured data.

9.5 Discussion and Lessons Learned

The architectural decision to decouple business logic into a Service Layer was highly effective, preventing code duplication and facilitating easier unit testing. The primary challenge was the complexity of data migration — extracting and structuring unstructured legacy content to fit the new relational database schemas required more effort than initially estimated. The mitigation strategy of developing robust database seeders with dummy data proved essential.

Chapter 10: Conclusion and Recommendations

10.1 Summary of the Project

The primary objective was to comprehensively modernize the SPU digital presence by engineering a highly performant, secure, and bilingual official website. By leveraging the Laravel MVC framework and the Filament CMS, the project fulfilled over 130 functional and non-functional requirements, successfully bridging the gap between SPU's academic prestige and its digital representation.

10.2 Key Conclusions

- **Administrative Autonomy:** The Filament CMS achieved "no-code" content governance. Faculty editors and administrators can seamlessly manage bilingual content, eliminating the operational bottleneck of relying on technical staff for daily updates.
- **Performance and Scalability:** Redis caching and Meilisearch full-text indexing resolved the legacy system's performance issues, processing up to 150 RPS with a mean response time well under 500ms.
- **Accessibility and Inclusivity:** Rigorous adherence to WCAG 2.1 Level AA standards and automated SEO generation ensures the university's resources are universally accessible and highly visible on global search engines.

10.3 Challenges Encountered

- **Data Migration Complexity:** Transitioning unstructured legacy content into the new relational MySQL 8 database required extensive data modeling, mitigated by automated database seeders.
- **Bilingual Synchronization:** Ensuring Arabic and English content maintained parity without layout breaks required custom frontend fallback logic.

10.4 Recommendations for Future Work

- **Mobile Application Integration:** Utilizing the Service Layer and Laravel's RESTful API capabilities to develop a dedicated cross-platform mobile application for enrolled students and faculty.
- **AI-Driven Admissions Assistant:** Integrating an NLP chatbot within the Admissions portal to provide prospective students with real-time, multilingual answers.
- **Advanced Analytics Integration:** Expanding the CMS to include granular web analytics, tracking engagement metrics, admission conversion rates, and visitor demographics.

